

AI Challenge Problem Statement: RupeeRadar

Background

Working professionals often make hundreds of monthly transactions across UPI, cards, bank transfers, subscriptions, EMIs, rent, shopping, food delivery, travel, and investments. Although bank statements contain all this information, they are difficult to understand because transaction descriptions are messy, inconsistent, and hard to categorize manually.

Challenge

Build **RupeeRadar**, an AI-powered personal finance assistant that helps users understand where their money is going by analyzing their bank statement data.

The application should allow a user to upload bank statement data, extract and clean transactions, categorize expenses, detect recurring payments, generate spending insights, and present the results in a simple dashboard or report.

Objective

Create an end-to-end solution that converts raw financial transaction data into meaningful personal finance insights.

The solution should help users answer questions such as:

- What are my biggest spending categories?
- How much did I spend this month?
- Which transactions are recurring subscriptions or EMIs?
- What was my biggest transaction?
- What are the top insights from my spending behavior?

Core Requirements

The solution should:

1. Accept bank statement data as input.
2. Extract or clean transactions into a structured format.
3. Categorize transactions into meaningful groups such as Food, Travel, Shopping, Bills, EMI, Subscriptions, Salary, Rent, Investments, and Other.
4. Identify recurring transactions such as subscriptions, EMIs, rent, SIPs, or insurance payments.
5. Calculate key financial metrics such as total income, total spend, savings, top categories, and biggest transactions.
6. Generate clear, human-readable spending insights using actual transaction amounts.
7. Present the final output through a simple user interface, dashboard, or downloadable report.

Expected Output

Participants should submit a working prototype that demonstrates:

- Cleaned transaction data
- Categorized expenses
- Recurring payment detection
- Spend summary dashboard
- At least three personalized financial insights
- A final report or visual summary that can be shared

Evaluation Criteria

Submissions will be evaluated on:

- Accuracy of transaction cleaning and categorization
- Quality of financial insights
- Ability to handle real-world messy transaction descriptions
- Simplicity and usefulness of the user experience
- Completeness of the end-to-end workflow
- Privacy-conscious handling of sensitive financial data

Constraint

The solution should prioritize a working end-to-end prototype over perfect support for every bank format. Participants may choose their own technology stack and implementation approach.

Final Deliverable

A deployed or locally runnable application that takes raw bank statement data and produces a clear personal finance summary showing where the user's money is going.

Introduction

Hi I am Saksham Arora, currently working as SDE 2 at Intuit, before 3 years I worked at Microsoft as a Software Engineer and did 2 internships at Microsoft as well.

My linkedin - <https://www.linkedin.com/in/sakshamarora9575/>

Youtube - <https://www.youtube.com/@saksham-arora>

AI Tools

1. <https://antigravity.google/product/antigravity-ide>
2. <https://qoder.com/download>
3. Cursor Download

STEPS

- **Context SETUP**
 - Docs folder
 - problemStatement.txt → basic problem statement
 - context.md → stores the necessary context of the project we are building
 - architecture.md → store the detailed architecture of the project
 - implementation-plan.md → store phase-wise implementation plan
 - edge-cases.md → corner scenarios or basic bugs which we can have during implementation
- **IMPLEMENTATION**
 - implementation-plan

Prompts

1. Generate a context.md file which stores the necessary context of this project using @docs/problemStatement.txt
2. Generate a detailed architecture.md using @docs/context.md
3. Generate an phase-wise implementation plan as per the @docs/architecture.md
4. Generate an edge-case.md file which will store the necessary corner cases for the project using @docs/implementation-plan.md and @docs/architecture.md
5. Implement phase0 as per the @docs/[implementation-plan.md](#)
LLM used in this project will be Groq
6. Implement phase1 as per the @docs/implementation-plan.md
7. Implement phase2 as per the @docs/implementation-plan.md
8. Generate sample data (csv) which we can upload on website and test till phase2 manually
9. Run the project
10. <ERROR>
500 internal server error is coming in the project please fix it
11. stop the project
12. Can we fix this pie chart or display a better as visuals are not that clear
13. Implement phase3 as per the @docs/implementation-plan.md
14. Run the project
15. Push the project on github → URL
16. Are all the deployment coding changes done ?

DEPLOYMENT

- <https://railway.com/> → BACKEND
- <https://vercel.com/> → FRONTEND

Architecture

RupeeRadar — System Architecture

This document describes the technical architecture for RupeeRadar, an AI-powered personal finance assistant. It is derived from [context.md](#) and defines how raw bank statement data flows through the system to produce categorized transactions, recurring-payment detection, financial metrics, and human-readable insights.

1. Architectural Goals

Goal	Rationale
End-to-end workflow	Upload → insights must work as a single user journey
Messy-data tolerance	Indian bank/UPI descriptions vary widely; normalization is first-class
Privacy by design	Financial data is sensitive; minimize exposure and retention
Prototype-first	Ship a working vertical slice before supporting every bank format
Inspectable output	Users should see cleaned data, not just a black-box summary
Extensible parsing	New bank formats plug in without rewriting the core pipeline

2. High-Level System View

RupeeRadar is organized as a **pipeline-oriented, modular monolith** for the prototype phase. A single application hosts ingestion, processing, analytics, and presentation. This keeps deployment simple while preserving clear module boundaries for future extraction into services.

Recommended Stack (Prototype)

Layer	Recommendation	Alternatives
Frontend	React + TypeScript + Tailwind	Next.js, Vue
Backend	Python FastAPI	Node.js Express, Django
Database	SQLite (local) → PostgreSQL (deployed)	—
File parsing	pandas + bank-specific parsers	openpyxl, pdfplumber

AI categorization	LLM API with structured output	Rule engine + LLM fallback
Charts	Recharts or Chart.js	D3
Report export	HTML → PDF (weasyprint / browser print)	—
Deployment	Docker + single VM / Railway / Render	Local-only demo

Stack choices are flexible per [context.md](#); the architecture below is stack-agnostic.

3. Core Domain Model

3.1 Entities

UploadSession

```

├── id (UUID)
├── filename, file_type, bank_hint (optional)
├── status: pending | parsing | processing | ready | failed
├── uploaded_at, expires_at (TTL for privacy)
└── error_message (if failed)

```

Transaction (cleaned, structured)

```

├── id
├── session_id
├── date
├── description_raw    # original text from statement
├── description_clean  # normalized merchant/description
├── amount             # signed: + credit, - debit
├── type: credit | debit
├── balance (optional)
├── category           # Food, Travel, ..., Other
├── category_confidence # 0.0–1.0
├── is_recurring       # boolean
├── recurring_group_id # links related recurring txns
└── metadata (JSON)    # UPI ref, mode, parser hints

```

RecurringGroup

```

├── id
├── session_id
├── label           # e.g. "Netflix", "Home Loan EMI"
├── category
├── frequency: weekly | monthly | quarterly | yearly | unknown
├── typical_amount
├── last_seen_date
├── transaction_ids[]
└── confidence

```

AnalysisResult

```

├── session_id
├── metrics (JSON)    # income, spend, savings, etc.
├── top_categories[]
├── biggest_transactions[]
├── insights[]        # human-readable strings
└── generated_at

```

3.2 Canonical Transaction Schema

All parsers must emit this normalized shape before downstream processing:

```
{
  "date": "2025-06-01",
  "description_raw": "UPI-SWIGGY-BANGALORE-123456",
  "amount": -450.00,
  "type": "debit",
  "balance": 12500.00
}
```

Normalization rules:

- Dates → ISO 8601 (YYYY-MM-DD)
- Amounts → decimal, INR, debits negative (or separate type + absolute amount — pick one convention and enforce consistently)
- Descriptions → trimmed; preserve description_raw for audit
- Duplicates → detect via (date, amount, description_raw) hash within a session

3.3 Category Taxonomy

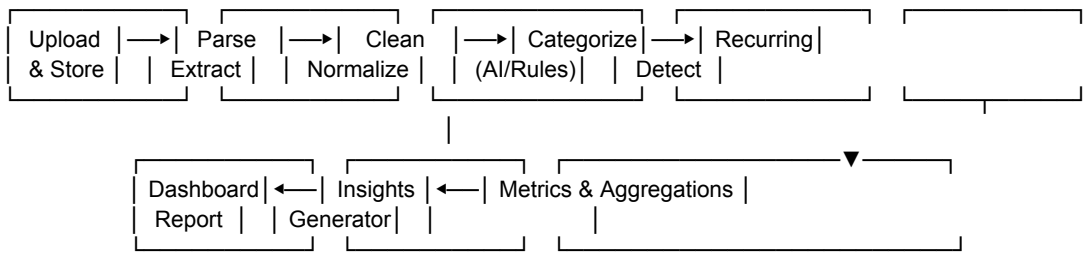
Fixed enum aligned with requirements:

Food · Travel · Shopping · Bills · EMI · Subscriptions · Salary · Rent · Investments · Other

Salary and Investments are typically credits; the rest are primarily debits. Other is the fallback when confidence is below threshold.

4. Processing Pipeline

The pipeline runs synchronously for small statements (< ~2,000 rows) and can move to a background job queue for larger files.



4.1 Stage 1 — Ingestion & Parsing

Responsibility: Accept files, detect format, extract raw rows.

Input format	Parser strategy
CSV	Column mapping via header detection + heuristics
Excel (.xlsx)	Same as CSV; handle merged cells

PDF	Table extraction (pdfplumber); bank-specific templates for prototype
-----	--

Parser interface (plugin pattern):

```
class StatementParser(Protocol):
    def can_parse(self, file: UploadFile) -> bool: ...
    def parse(self, file: UploadFile) -> list[RawTransaction]: ...
```

Prototype scope: Support 1–2 common Indian bank CSV exports (e.g. HDFC, ICICI, SBI). Generic CSV mapper with user column mapping as fallback.

Validation:

- Reject empty files, password-protected PDFs (with clear error)
- Enforce max file size (e.g. 10 MB)
- Log parse warnings (skipped rows, ambiguous dates) for UI display

4.2 Stage 2 — Cleaning & Normalization

Responsibility: Turn messy descriptions into structured, comparable text.

Step	Example
Strip noise	UPI/DR/123456/SWIGGY → SWIGGY
Merchant extraction	Regex + known-merchant dictionary (Swiggy, Zomato, Amazon, etc.)
Mode detection	UPI, NEFT, IMPS, card, cash
Amount sign correction	Ensure debits/credits align with statement conventions
Date repair	Handle DD-MM-YYYY vs DD/MM/YY

Output: description_clean, optional merchant, payment_mode.

Cleaning is largely **deterministic** (rules + dictionary). This improves categorization accuracy and keeps LLM calls smaller.

4.3 Stage 3 — Categorization

Responsibility: Assign each transaction to the taxonomy.

Hybrid approach (recommended):

1. Rule engine (fast, high precision for known patterns)
↓ unmatched
2. Merchant dictionary lookup
↓ still unmatched
3. LLM batch categorization (structured JSON output)
↓ low confidence
4. Default → Other

Rule examples:

Pattern	Category
SALARY, NEFT CREDIT SALARY	Salary
NETFLIX, SPOTIFY, YOUTUBE	Subscriptions
SWIGGY, ZOMATO, DOMINOS	Food
UBER, OLA, IRCTC, MAKEMYTRIP	Travel
SIP, ZERODHA, GROWW	Investments
HOME LOAN, EMI	EMI

LLM prompt design:

- Batch transactions (e.g. 20–50 per call) with description_clean, amount, type
- Request JSON: { "id": "...", "category": "...", "confidence": 0.85 }
- Include few-shot examples for Indian UPI-style strings
- Never send account numbers or full raw statements if avoidable

Confidence threshold: e.g. $< 0.6 \rightarrow$ Other, flag for user review in UI.

4.4 Stage 4 — Recurring Payment Detection

Responsibility: Identify subscriptions, EMIs, rent, SIPs, insurance.

Heuristic algorithm:

1. Group debits by similar description_clean (fuzzy match, Levenshtein or token overlap)
2. Within each group, check:
 - ≥ 2 occurrences
 - Amount within tolerance (e.g. $\pm 5\%$ or $\pm ₹50$)
 - Intervals consistent with monthly (~28–32 days) or other cadence
3. Label group from merchant name or dominant keyword
4. Override category if pattern matches known recurring types (EMI, SIP)

Output: RecurringGroup records + is_recurring flags on transactions.

4.5 Stage 5 — Metrics & Aggregations

Responsibility: Compute summary numbers for dashboard and insights.

Metric	Definition
Total income	Sum of credits in selected period
Total spend	Sum of debits (absolute value)
Savings	Income – spend
Savings rate	$\text{Savings} / \text{income} \times 100$
Top categories	Debit totals grouped by category, sorted desc

Biggest transaction	Single largest debit (and optionally credit)
Monthly spend	Debits grouped by calendar month
Recurring total	Sum of typical amounts from recurring groups

Time scope: Default to full statement range; UI filter for "this month" based on latest transaction date or user selection.

4.6 Stage 6 — Insight Generation

Responsibility: Produce ≥ 3 personalized, human-readable insights using real amounts.

Two-tier approach:

- Template insights (deterministic, always available):**
 - "You spent ₹X on Food this month — your largest category."
 - "Your biggest transaction was ₹Y to {merchant} on {date}."
 - "We detected N recurring payments totalling ₹Z/month."
- LLM-enhanced insights (optional):**
 - Feed metrics + top categories + recurring summary (not raw PII)
 - Generate 2–3 narrative observations (spending trends, category shifts)

Insights must cite **actual transaction amounts** per requirements.

4.7 Stage 7 — Presentation & Export

Dashboard views:

View	Content
Summary cards	Income, spend, savings, savings rate
Category breakdown	Pie/bar chart by category
Monthly trend	Line/bar chart of spend over time
Transaction table	Sortable, filterable, with category badges
Recurring panel	List of detected subscriptions/EMIs with amounts
Insights panel	≥ 3 insight cards

Report export:

- HTML report rendered server-side or client-side
- PDF via print stylesheet or weasyprint
- Shareable summary (screenshot-friendly single page)

5. API Design

RESTful API under /api/v1. All endpoints scoped to an UploadSession.

Method	Endpoint	Description
POST	/upload	Upload statement file; returns session_id
GET	/sessions/{id}	Session status and metadata
GET	/sessions/{id}/transactions	Paginated cleaned transactions
PATCH	/sessions/{id}/transactions/{txn_id}	User override category (optional)
GET	/sessions/{id}/recurring	Detected recurring groups
GET	/sessions/{id}/analytics	Metrics and aggregations
GET	/sessions/{id}/insights	Generated insights
GET	/sessions/{id}/report	Downloadable report (HTML/PDF)
DELETE	/sessions/{id}	Purge session data

Upload flow:

POST /upload (multipart file)
 → 202 { session_id, status: "processing" }
 → client polls GET /sessions/{id} until status = "ready"
 → client fetches transactions, analytics, insights in parallel

For prototype simplicity, processing can be synchronous with a loading spinner for small files.

6. Frontend Architecture

6.1 Page Structure

/ → Landing + upload
 /analysis/:sessionId → Main dashboard (tabs or sections)

- ├── Summary
- ├── Transactions
- ├── Recurring
- ├── Insights
- └── Export

6.2 State Management

- **Server state:** React Query / SWR for API data (transactions, analytics)
- **UI state:** Local component state (filters, sort, active tab)
- **No persistent client storage** of financial data (privacy)

6.3 Key Components

Component	Responsibility
FileUpload	Drag-drop, format hints, progress
ProcessingStatus	Pipeline stage indicator
SummaryCards	Income, spend, savings
CategoryChart	Visual breakdown
TransactionTable	Search, filter by category, edit category
RecurringList	Recurring payment cards
InsightCards	Narrative insights
ReportExport	Download / print trigger

6.4 UX Principles

- Show cleaned data alongside raw description (transparency)
- Allow category override (improves trust and future rule learning)
- Clear errors for unsupported formats with sample CSV template
- Mobile-responsive but desktop-first for tables

7. Data Storage

7.1 Prototype Schema (relational)

upload_sessions
transactions
recurring_groups
analysis_results

SQLite suffices for local demo. PostgreSQL for deployment.

7.2 Retention Policy

Policy	Implementation
Session TTL	Auto-delete sessions after 24–72 hours (configurable)
No cross-user data	Sessions are anonymous or single-user
Upload files	Delete raw file after successful parse
LLM logs	Do not log full transaction descriptions externally

7.3 Indexing

- transactions(session_id, date)
- transactions(session_id, category)
- recurring_groups(session_id)

8. Security & Privacy Architecture

Privacy boundaries	
Browser —HTTPS—> API (auth optional for prototype)	
API —local DB—> No third-party storage of statements	
API —minimal fields—> LLM (only if user consents)	
Raw files deleted post-parse	
Session purge via TTL or explicit DELETE	

Control	Prototype	Production hardening
Transport	HTTPS	TLS 1.2+
Authentication	Optional / none	OAuth or magic link
File validation	Type + size checks	Virus scan, content sniffing
Secrets	.env, not committed	Secret manager
LLM data	Batch descriptions only	DPA, opt-out, local model option
CORS	Restrict to frontend origin	Same

9. Error Handling & Observability

9.1 Failure Modes

Stage	User-facing message	Recovery
Unsupported format	"We couldn't read this file. Try CSV export from your bank."	Offer sample template
Parse partial failure	"Imported 450 of 462 rows"	Show warnings list
Empty statement	"No transactions found"	Re-upload
LLM unavailable	Fallback to rules-only categorization	Degrade gracefully
Processing timeout	"Still processing..." or retry	Background job + poll

9.2 Logging

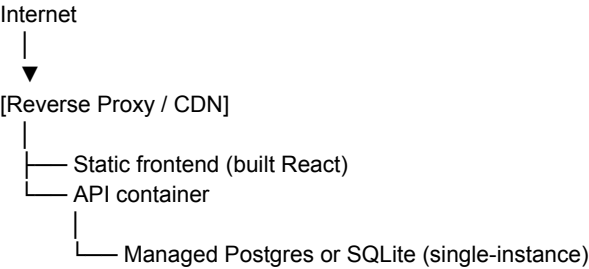
- Structured logs: session_id, stage, duration_ms, row_count
- **Never log** full account numbers, balances, or raw file contents
- Metrics: parse success rate, categorization confidence distribution, p95 latency

10. Deployment Architecture

10.1 Local Development

```
docker compose up
├── api (FastAPI, port 8000)
├── web (Vite dev server, port 5173)
└── db (SQLite volume or Postgres)
```

10.2 Prototype Deployment



Single-container deployment is acceptable for demo/evaluation.

10.3 Environment Variables

```
DATABASE_URL
LLM_API_KEY      # optional
SESSION_TTL_HOURS
MAX_UPLOAD_SIZE_MB
CORS_ORIGINS
```

11. Testing Strategy

Layer	Focus
Unit	Parsers, cleaners, categorization rules, recurring detector, metrics
Integration	Full pipeline on fixture CSVs (messy real-world samples)
Golden files	Expected output for 3–5 anonymized statements
E2E	Upload → dashboard shows categories + insights
Manual	UX walkthrough, report export, mobile layout

Fixture requirements: Include messy UPI strings, mixed date formats, salary credits, EMI debits, and duplicate-like entries.

12. Implementation Phases

Phase 1 — Vertical Slice (MVP)

- ☐ CSV upload for one bank format
- ☐ Parse → clean → rule-based categorize
- ☐ Basic metrics (income, spend, savings, top categories)
- ☐ Simple dashboard with transaction table
- ☐ 3 template-based insights

Phase 2 — Intelligence & Detection

- ☐ LLM categorization fallback
- ☐ Recurring payment detection
- ☐ Monthly trend chart
- ☐ Category override in UI

Phase 3 — Polish & Deliverable

- ☐ Second bank format or generic column mapper
- ☐ PDF report export
- ☐ Session TTL and DELETE endpoint
- ☐ Deployed demo URL
- ☐ LLM-enhanced narrative insights

13. Mapping to Requirements

context.md requirement	Architecture component
Accept bank statement data	Ingestion module, POST /upload
Extract/clean transactions	Parser plugins + Cleaning stage
Categorize expenses	Rule engine + LLM hybrid
Detect recurring payments	Recurring detection stage
Calculate metrics	Analytics module
Human-readable insights	Insight generator (templates + LLM)
Dashboard / report	Frontend views + /report export
Privacy-conscious handling	TTL, no raw file retention, minimal LLM payload
Prototype over perfect bank support	Plugin parsers, 1–2 formats first

14. Open Decisions

Decision	Options	Recommendation
Debit sign convention	Negative amounts vs. type + positive	Negative for debits (simpler summing)
Auth for prototype	None vs. simple password	None; add note in demo
LLM provider	OpenAI, Anthropic, local	Whichever is available; abstract behind interface
PDF parsing	Generic vs. bank-specific	CSV first; PDF as stretch
Background jobs	Sync vs. Celery/RQ	Sync until >5s processing observed

15. Directory Structure (Proposed)

```

rupee-radar/
├── docs/
│   ├── context.md
│   ├── problemStatement.txt
│   └── architecture.md    # this file
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/routes/
│   │   ├── parsers/      # bank-specific parsers
│   │   ├── pipeline/
│   │   │   ├── cleaner.py
│   │   │   ├── categorizer.py
│   │   │   ├── recurring.py
│   │   │   ├── metrics.py
│   │   │   └── insights.py
│   │   ├── models/
│   │   └── services/
│   ├── tests/
│   │   └── fixtures/     # anonymized sample statements
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── api/
│   └── package.json
├── docker-compose.yml
└── README.md

```

This architecture prioritizes a complete, demonstrable workflow. Depth can be added incrementally to parsers and categorization without changing the overall shape of the system.